

Reinforcement Learning

Assignment 2 - Lunar Lander

Authors:

Martin Nilsson, *School of Engineering*, Jönköping University

Besan Ewir, *School of Engineering*, Jönköping University

Vladimir Giustacchini, *School of Engineering*, Jönköping University

Jorge Cuello, *School of Engineering*, Jönköping University

September 23, 2026

Contents

1	Theoretical Part	2
1.1	Question 1	2
1.2	Question 2	2
1.3	Question 3	2

1 Theoretical Part

1.1 Question 1

Q1: “Explain the concept of the Q-learning update rule. How does it relate to the Bellman optimality equation? What are the potential challenges in using this update rule? “

1.2 Question 2

Q2: “Discuss the role of experience replay in DQN. How does it help to address the correlation problem in reinforcement learning? “

Experience Replay is one of the critical elements of Deep Q-Networks (DQN), where the agent saves its experiences (s, a, r, s') in the memory, which can be used during learning rather than learning from the latest experience only. In the training process, samples are randomly selected from the experience replay buffer.

The main benefit of using experience replay is that it helps reduce the correlation problem, since in reinforcement learning, the experiences that come one after another are highly correlated. Random sampling mixes experiences from different time steps, which reduces the correlation between successive updates and makes learning more stable.

Experience Replay also allows the same experience to be used multiple times while the agent still learning, making learning more efficient and reducing the variance of the updates.

1.3 Question 3

Q3: “Explain the concept of the target network in DQN. How does it help to stabilize the learning process? “

Target Network:

The target network is a copy of the Q-network that is used to calculate the target Q-values during training. The target network is updated less frequently than the Q-network, which helps to stabilize the learning process by reducing the correlation between the Q-values and the target values.

Keep in mind that the *target network* is **NOT** necessarily a different architecture or a separate trained network. In most cases it is simply a periodically updated copy of the online network.

In a neural network with θ parameters, we have $Q(s, a; \theta)$ and the target becomes

$$y = r + \gamma \max_{a'} Q(s', a'; \theta)$$

But since we use the same network to calculate the target, it can lead to instability and divergence in the learning process because the target would move with the parameters. To address this issue, we introduce a *target network* with parameters θ^- , which is a copy of the Q-network that is updated less frequently (e.g., every C steps). The target becomes

$$y = r + \gamma \max_{a'} Q(s', a'; \theta^-)$$

- $Q(s, a; \theta)$ (**Online network** → the network that is being trained)
- $Q(s', a'; \theta^-)$ (**Target network** → the network that is used to calculate the target Q-values)

A simplified DQN training loop with a target network can be described as follows:

DQN Target-Network Pseudocode

Initialize online network θ

Initialize target network $\theta^- = \theta$

Repeat:

1. Take an action using the online network.
2. Store transition (s, a, r, s') in the Experience/Replay buffer.
3. Sample a mini-batch from the replay buffer.
4. Calculate targets using the target network:

$$y = r + \gamma \max_{a'} Q(s', a'; \theta^-).$$

5. Update the online network θ to minimize:

$$\left(y - Q(s, a; \theta)\right)^2.$$

6. Every C steps, update the target network:

$$\theta^- \leftarrow \theta.$$

The main takeaway is that the target network helps to stabilize the learning process by providing a more stable target for the Q-value updates, which reduces the risk of divergence and improves convergence.